浙江大学实验报告

专业：电子科学与技术
姓名：郑智瑄
学号：3230103913
日期：April 3, 2026
地点：玉泉高分子楼-203

课程名称：人工智能实验　指导老师：魏准　实验类型：设计型实验
实验名称：Nonlinear SVM & PCA　成　绩：　签　名：

装 订 线

一、 SVM 非线性分类 II

1. 实验目的

使用 `make_moons` 生成的月牙形非线性数据集，通过 Scikit-Learn 的 Pipeline 构建多项式核 SVM 分类器，系统考察：（1）正则化参数 C（取 1、10、100）对分类效果的影响；（2）多项式阶数（$d = 2, 3, 5, 10$）对决策边界复杂度的影响；（3）不同核函数（线性、多项式、RBF、Sigmoid）在同一非线性数据集上的性能对比。

2. 实验原理

多项式核函数定义为：

$$K(\mathbf{x}_i, \mathbf{x}_j) = (1 + \langle \mathbf{x}_i, \mathbf{x}_j \rangle)^d$$

其中 d 为多项式次数（`degree`），偏置项 $r = 1$（`coef0`）。多项式核通过升维隐式计算高次特征之间的内积，避免了显式映射的高计算代价，适用于低维输入、特征交互重要的非线性问题。与 RBF 核相比，多项式核具有全局性（每对样本均相互影响），而 RBF 核则具有局部性（仅近邻样本权重显著）。

核 SVM 的对偶目标函数将线性内积 $\langle \mathbf{x}_i, \mathbf{x}_j \rangle$ 替换为核函数 $K(\mathbf{x}_i, \mathbf{x}_j)$，从而在原始空间中隐式完成高维分类，无需显式计算特征映射 $\phi(\mathbf{x})$。

Pipeline 工作流　实验代码使用 Scikit-Learn 的 `Pipeline` 将数据预处理与模型训练串联为一个整体，确保训练与预测阶段使用完全相同的变换参数：

$$\text{原始特征} \xrightarrow{\texttt{StandardScaler}} \text{标准化特征（零均值、单位方差）} \xrightarrow{\texttt{SVC(kernel="poly"}, d, C)} \text{分类结果}$$

其中 `StandardScaler` 对各特征减去训练集均值并除以标准差，消除量纲差异对 SVM 间隔计算的干扰；`SVC` 随后在标准化空间中以多项式核求解最优分类超平面。`Pipeline` 的关键优势在于：（1）避免数据泄露——Scaler 的均值与标准差仅从训练集估计，预测时对测试集应用相同参数；（2）简化超参数搜索——整个 Pipeline 可直接传入 `GridSearchCV`，使预处理与分类器超参数的联合调优更加规范。

3. 核心代码实现

```python
import numpy as np
from sklearn.datasets import make_moons
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

# (1) 产生非线性数据集
X, y = make_moons(n_samples=100, noise=0.15, random_state=42)

# (2) 利用 Pipeline 构建多项式核 SVM（以 C=10, degree=3 为例）
polynomial_kernel_svm_clf = Pipeline([
    ("scaler", StandardScaler()),                # (3) 标准化数据
    ("svm_clf", SVC(kernel="poly", degree=3,     # (4) 多项式核 SVC
                    coef0=1, C=10, random_state=42)),
])
# (5) 训练模型
polynomial_kernel_svm_clf.fit(X, y)

# (7) 测试数据
test_pts = np.array([[0.0, 0.0], [1.0, 0.5], [-0.5, -0.3]])
print(polynomial_kernel_svm_clf.predict(test_pts))
```

Listing 1: SVM 非线性分类 II 核心代码（节选）

4. 运行结果

(1) 讨论 C 值的影响（多项式核，$d = 3$）

```
多项式核 SVM (degree=3) -- 不同 C 值
  C=   1   accuracy=0.9600 test_pts predict: [1 0 1]
  C=  10   accuracy=0.9900 test_pts predict: [1 0 1]
  C= 100   accuracy=0.9900 test_pts predict: [1 0 1]
```

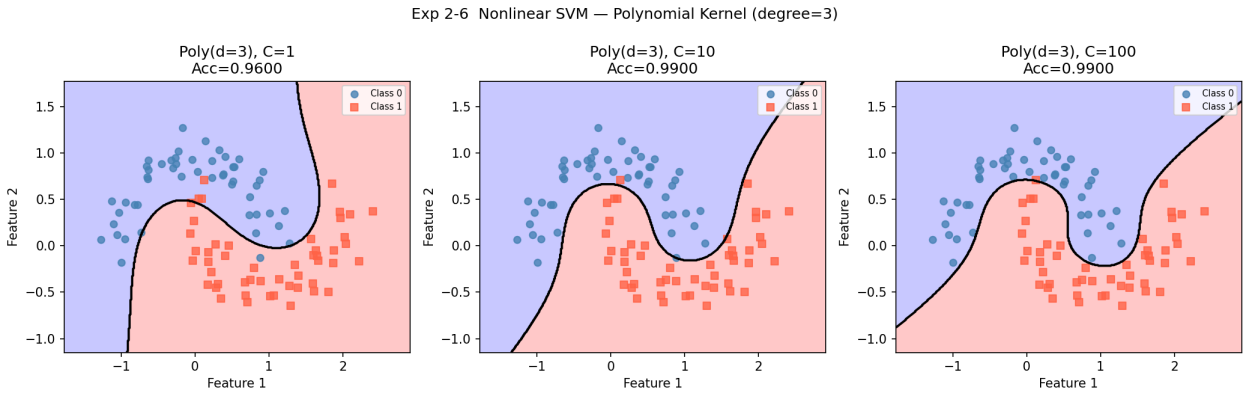


Figure 1: SVM 非线性分类 II——多项式核（$d = 3$）在不同 C 值（1、10、100）下的决策边界

(2) 讨论不同多项式阶数（$C = 5$）

```
多项式核 SVM (C=5) -- 不同 degree
  degree= 2 accuracy=0.8700
  degree= 3 accuracy=0.9800
  degree= 5 accuracy=0.9900
  degree=10 accuracy=1.0000
```

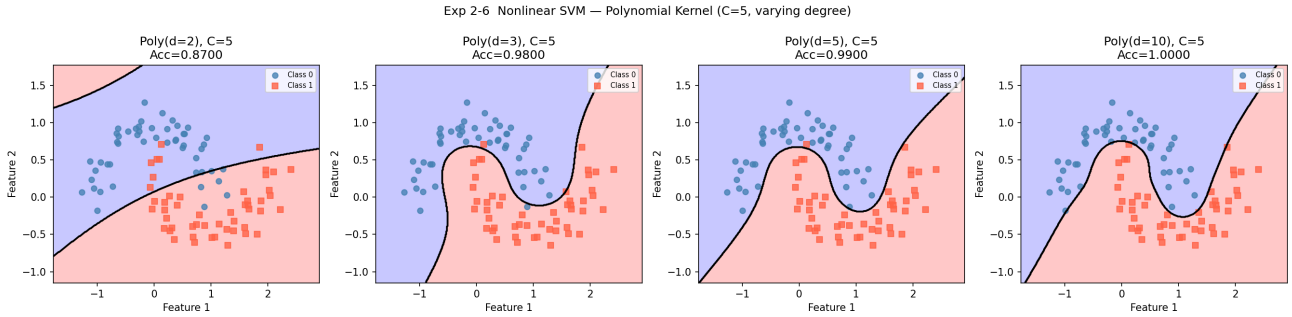


Figure 2: SVM 非线性分类 II——不同多项式阶数（$d = 2, 3, 5, 10$，$C = 5$）的决策边界

(3)　不同核函数对比（$C = 5$）

```
不同核函数对比（C=5）
  Linear (C=5)          accuracy=0.8600
  Poly(d=3, C=5)        accuracy=0.9800
  RBF (C=5, gamma=0.5)  accuracy=0.9800
  Sigmoid (C=5)         accuracy=0.7100
```

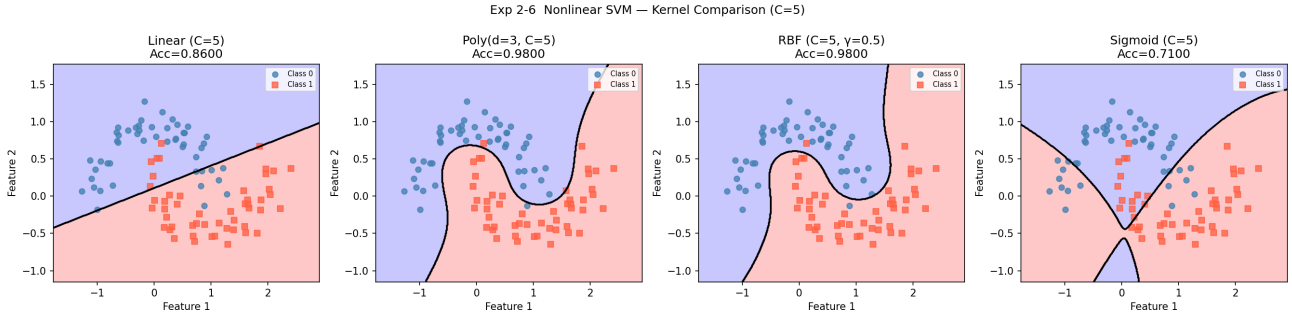


Figure 3: SVM 非线性分类 II——四种核函数（线性、多项式、RBF、Sigmoid）决策边界对比（$C = 5$）

5. 结果分析

C 值的影响　在三次多项式核下，$C = 1$ 时准确率为 0.9600；$C = 10$ 和 $C = 100$ 时均达到 0.9900，提升趋于饱和。较小的 C 允许更多间隔违例，决策边界更平滑；C 增大后模型更严格拟合训练集，曲率略有增加，如图 1 所示。对月牙形数据而言，$C \in [10, 100]$ 已能取得足够好的拟合，继续增大 C 提升有限。

多项式阶数的影响　如图 2 所示，随着 d 增大，决策边界的弯曲程度和复杂度递增：$d = 2$ 时准确率仅 0.87，低阶多项式无法刻画月牙形的强非线性；$d = 3$ 提升至 0.98，已能较好匹配数据分布；$d = 5$ 进一步达到 0.99；$d = 10$ 达到 1.00，但对应的决策边界已出现过拟合迹象（边界局部扭曲）。实践中应在验证集上选取合适的 d，而非一味提高阶数。

不同核函数对比 如图 3 所示：

- **线性核**（0.86）：月牙形数据线性不可分，直线决策面误分类较多；
- **多项式核**（0.98）和 **RBF 核**（0.98）：均能学得与数据分布吻合的曲线决策边界，性能相当；
- **Sigmoid 核**（0.71）：在本数据集上表现最差，Sigmoid 核在某些参数设置下可能导致不定核矩阵，鲁棒性较低。

综合来看，多项式核与 RBF 核均适合月牙形数据，实际应用中推荐优先测试 RBF 核（超参数少），再考虑多项式核。

装 订 线

二、 PCA 降维——实验 3-1

1. 实验目的

基于课件给出的 PCA 算法框架，补全 `lowDDataMat` 与 `reconMat` 的求解方式；对 `testSet.txt`（1000 个二维样本点）分别取 `topNfeat`=1 和 2 进行降维，可视化降维结果，并通过重建误差与图形对比讨论两种情形的降维效果。

2. 实验原理

PCA（主成分分析）的目标是找到一组正交基，使得数据在该基上的投影方差最大（信息保留最多）。其核心步骤为：

(1) **去除均值**：$\tilde{X} = X - \bar{X}$；

(2) **计算协方差矩阵**：$C = \frac{1}{m}\tilde{X}^\top \tilde{X}$；

(3) **特征值分解**：$C\mathbf{v} = \lambda\mathbf{v}$，取前 N 个最大特征值对应的特征向量 $W \in \mathbb{R}^{d\times N}$；

(4) **降维**：$Z = \tilde{X}W$（N 维新空间）；

(5) **重建**：$\hat{X} = ZW^\top + \bar{X}$。

当 $N < d$ 时，$\hat{X}$ 是原始数据在前 N 个主成分构成的子空间上的投影，重建误差即为被舍弃的低方差成分之和。

为何取协方差矩阵特征向量为基——方差最大化推导　设去均值后的数据矩阵为 $\tilde{X} \in \mathbb{R}^{m\times d}$，第 i 个样本为 $\tilde{\mathbf{x}}_i \in \mathbb{R}^d$。寻找单位方向向量 $\mathbf{w} \in \mathbb{R}^d$（$\|\mathbf{w}\| = 1$），使所有样本在 $\mathbf{w}$ 上的投影坐标 $\tilde{\mathbf{x}}_i^\top \mathbf{w}$ 的方差最大：

$$D(\mathbf{w}) = \frac{1}{m}\sum_{i=1}^{m}\left(\tilde{\mathbf{x}}_i^\top \mathbf{w}\right)^2 = \frac{1}{m}\sum_{i=1}^{m}\mathbf{w}^\top\tilde{\mathbf{x}}_i\tilde{\mathbf{x}}_i^\top\mathbf{w} = \mathbf{w}^\top\underbrace{\left(\frac{1}{m}\sum_{i=1}^{m}\tilde{\mathbf{x}}_i\tilde{\mathbf{x}}_i^\top\right)}_{\Sigma\ (\text{协方差矩阵})}\mathbf{w} = \mathbf{w}^\top\Sigma\,\mathbf{w}$$

带等式约束 $\mathbf{w}^\top\mathbf{w} = 1$ 的优化问题写为 Lagrangian：

$$\mathcal{L}(\mathbf{w},\lambda) = \mathbf{w}^\top\Sigma\,\mathbf{w} + \lambda\left(1 - \mathbf{w}^\top\mathbf{w}\right)$$

对 $\mathbf{w}$ 求导并令其为零：

$$\frac{\partial\mathcal{L}}{\partial\mathbf{w}} = 2\Sigma\,\mathbf{w} - 2\lambda\mathbf{w} = \mathbf{0} \quad\Longrightarrow\quad \Sigma\,\mathbf{w} = \lambda\mathbf{w}$$

即 $\mathbf{w}$ 为协方差矩阵 Σ 的**特征向量**，λ 为对应特征值。此时目标值为：

$$D(\mathbf{w}) = \mathbf{w}^\top\Sigma\,\mathbf{w} = \mathbf{w}^\top(\lambda\mathbf{w}) = \lambda\,\|\mathbf{w}\|^2 = \lambda$$

因此，投影方差等于特征值本身。要使方差最大，应取**最大特征值**对应的特征向量作为第一主成分方向；依次取次大特征值对应的特征向量（且相互正交），即得前 N 个主成分，构成投影矩阵 $W \in \mathbb{R}^{d\times N}$。这正是 PCA 步骤 3 中对协方差矩阵做特征值分解、并按特征值降序排列取前 N 列的理论依据。

装 订 线

3. 核心代码实现

```python
from numpy import mean, cov, linalg
import numpy as np
mat     = np.matrix
argsort = np.argsort

def loadDataSet(fileName, delim='\t'):
    fr = open(fileName)
    stringArr = [line.strip().split(delim) for line in fr.readlines()]
    datArr    = [list(map(float, line)) for line in stringArr]
    return mat(datArr)

def pca(dataMat, topNfeat=9999999):
    meanVals     = mean(dataMat, axis=0)
    meanRemoved = dataMat - meanVals            # remove mean
    covMat       = cov(meanRemoved, rowvar=0)
    eigVals, eigVects = linalg.eig(mat(covMat))
    eigValInd    = argsort(eigVals)             # sort: small -> large
    eigValInd    = eigValInd[:-(topNfeat+1):-1]  # keep top N
    redEigVects = eigVects[:, eigValInd]

    # (1) 将数据投影到 N 维新空间
    lowDDataMat = meanRemoved * redEigVects
    # (2) 从低维空间恢复到原始空间
    reconMat    = (lowDDataMat * redEigVects.T) + meanVals
    return lowDDataMat, reconMat
```

Listing 2: PCA 函数完整实现

4. 运行结果

程序输出如下：

```
原始数据维度: (1000, 2) (样本数=1000, 特征数=2)

[topNfeat=1]
  降维后数据形状: (1000, 1)
  恢复数据形状 : (1000, 2)
  降维后坐标（前5行）:
    [[-2.51033597]
     [-2.86915379]
     [ 0.09741085]
     [-0.76778222]
     [ 1.02715333]]
  均方重建误差 : 0.3661

[topNfeat=2]
  降维后数据形状: (1000, 2)
  恢复数据形状 : (1000, 2)
  均方重建误差 : 0.0000
```

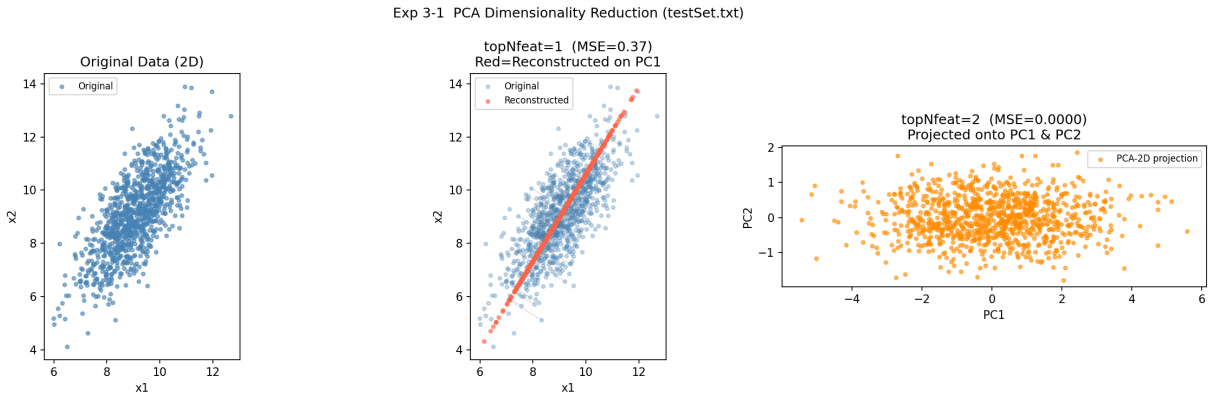


Figure 4: PCA 降维结果——左：原始 2D 数据；中：`topNfeat=1` 的重建点（红色，投影至 PC1 直线）与原始数据（蓝色）对比；右：`topNfeat=2` 在 PC1–PC2 坐标系下的分布

5. 结果分析

topNfeat=1 的降维效果　保留第一主成分（PC1）时，原 2D 数据被压缩到 1D（每个样本只剩一个坐标值）。重建时将 1D 坐标沿 PC1 方向反投影回 2D，得到的恢复点均落在 PC1 所在的直线上（图 4 中图红色点），与原始蓝色点之间存在垂直偏差。均方重建误差为 **0.3661**，对应第二主成分（PC2）方向上的方差损失。由于 testSet.txt 数据沿 PC1 方向具有较大展开、沿 PC2 方向展开较小，1 维保留了数据的主要结构，但仍有一定精度损失。

topNfeat=2 的降维效果　保留两个主成分时，原始 2D 数据被投影到新的正交坐标系（PC1，PC2）下（图 4 右图），仅做了旋转变换，信息无任何损失，均方重建误差为 **0.0000**。新坐标系下 PC1 轴对应最大方差方向，PC2 轴对应次大方差方向且与 PC1 正交，数据的整体形态与原始分布一致，只是坐标轴发生了旋转。

综合对比　对于 $d = 2$ 的数据集，`topNfeat=2` 等价于无损坐标旋转；`topNfeat=1` 则是真正的降维（$2 \to 1$），以 0.3661 的重建误差换取了一半的存储/计算开销。若数据本身在第二主成分方向上方差极小，1 维降维的信息损失可以接受；对于需要精确重建的场景则应保留更多主成分。

装 订 线

topNfeat 装 订

三、 PCA 降维——实验 3-2

1. 实验目的

利用 PCA 对半导体制造数据集 `secom.data`（1567 样本，590 特征，含大量 NaN）进行降维：（1）讨论 `topNfeat` 取值对降维数据的影响；（2）找到使降维重建数据与原始数据的**相对误差小于 9%** 的最小 `topNfeat` 值。

相对误差定义为：

$$\text{Re_err} = \frac{\|X - \hat{X}\|_F}{\|X\|_F}$$

2. 实验原理

半导体制造数据包含大量 NaN 缺失值，需先用各列非 NaN 元素的均值填充，再执行 PCA。数据共 590 个特征，PCA 对协方差矩阵做特征值分解，按特征值从大到小排列后保留前 N 个主成分；N 越大，重建精度越高，但压缩比越低。通过扫描不同 N 并计算 Re_err，可找到精度需求与压缩效率的平衡点。

3. 核心代码实现

```python
from numpy import mean, cov, linalg, isnan, nonzero, shape
import numpy as np
mat = np.matrix

def replaceNanWithMean(fileName):
    datMat  = loadDataSet(fileName, delim=' ')
    numFeat = shape(datMat)[1]
    for i in range(numFeat):
        # 用非 NaN 列均值填充缺失值
        meanVal = mean(datMat[nonzero(~isnan(datMat[:, i].A))[0], i])
        datMat[nonzero(isnan(datMat[:, i].A))[0], i] = meanVal
    return datMat

# 扫描 topNfeat，计算相对误差
datMat    = replaceNanWithMean('secom.data')
norm_orig = linalg.norm(datMat)
for N in [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 15, 20, ...]:
    _, reconMat = pca(datMat, topNfeat=N)
    re_err = linalg.norm(datMat - reconMat) / norm_orig
    print(f"topNfeat={N:4d}   Re_err={re_err*100:.3f}%")
```

Listing 3: 实验 3-2 核心代码——NaN 填充与相对误差扫描

4. 运行结果

```
数据维度: 1567 样本 × 590 特征

topNfeat=  1   Re_err=34.212%
topNfeat=  2   Re_err=21.851%
topNfeat=  3   Re_err=14.651%
topNfeat=  4   Re_err=12.188%
topNfeat=  5   Re_err=10.327%
topNfeat=  6   Re_err= 9.578%
```

```
topNfeat= 7   Re_err= 9.081%
topNfeat= 8 Re_err= 8.569% <-- 首次低于 9%topNfeat= 9   Re_err= 8.116%
topNfeat= 10  Re_err= 7.696%
topNfeat= 15  Re_err= 5.925%
topNfeat= 20  Re_err= 4.572%
topNfeat= 50  Re_err= 0.843%
topNfeat= 100 Re_err= 0.145%
topNfeat= 200 Re_err= 0.008%

相对误差 < 9% 的最小 topNfeat = 8 (Re_err=8.5691%)
```

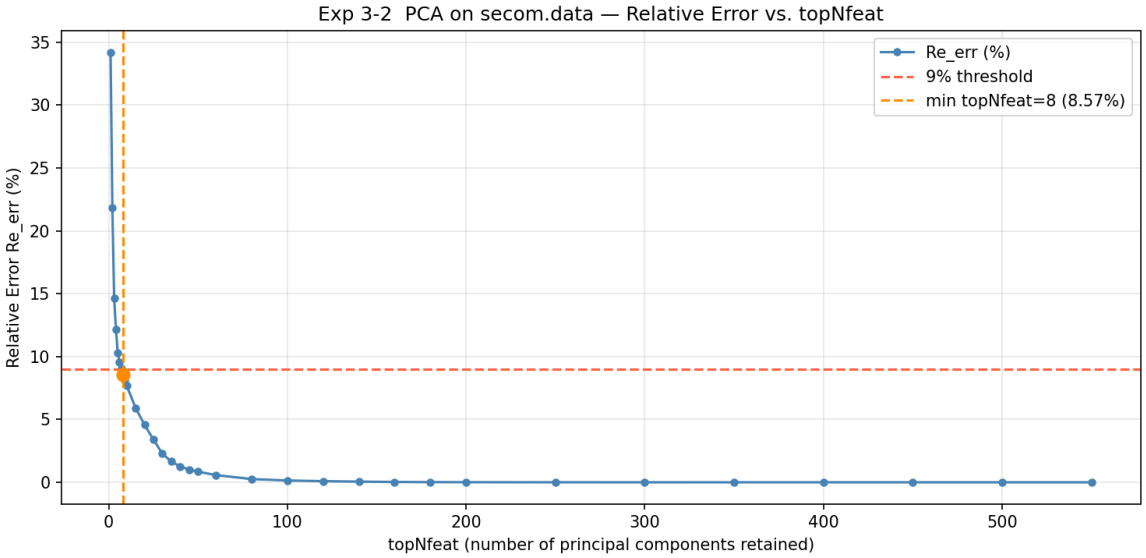


Figure 5: 实验 3-2——相对误差随 `topNfeat` 的变化曲线；红色虚线为 9% 阈值，橙色竖线标注最小满足点 $N = 8$

5. 结果分析

topNfeat 对降维数据的影响　如图 5 所示，相对误差随主成分数 N 的增大**单调递减**，符合 PCA 按方差贡献从大到小保留主成分的理论预期。

- $N = 1 \sim 3$：误差迅速从 34.2% 降至 14.7%，前几个主成分捕获了数据中大量的高方差特征，每增加一个主成分都带来显著的精度提升；
- $N = 4 \sim 10$：进入较平缓的下降段，误差从 12.2% 降至 7.7%，说明后续主成分的单个方差贡献在递减；
- $N > 50$：曲线趋近于零，增加更多主成分的边际效益极小；
- $N = 590$（全部特征）：误差降至机器精度，等价于无损 PCA 旋转。

最小 topNfeat 结论　满足相对误差 $< 9\%$ 的**最小 topNfeat 为 8**，对应 Re_err = 8.57%。这意味着仅用 590 个特征中最重要的 **8 个主成分**（压缩比约 1.4%），即可将数据重建误差控制在 9% 以内，体现了半导体制造数据中少数特征主导了数据绝大部分方差的特性。实际应用中可根据具体精度需求在 $N = 8$（高压缩）与较大的 N 之间权衡。

四、 实验总结

本次实验涵盖 SVM 非线性分类与 PCA 降维两个主题，分别从判别模型与无监督特征提取两个角度加深了对经典机器学习算法的理解。

SVM 非线性分类 II 在月牙形数据集上系统考察了多项式核 SVM 的关键超参数。正则化参数 C 增大可提升训练精度，但边际效益递减，$C \geq 10$ 后性能趋于饱和；多项式阶数 d 过高时决策边界出现局部扭曲，存在明显过拟合风险。四种核函数的横向对比表明，多项式核与 RBF 核性能相当（均达 0.98），线性核因数据线性不可分表现较差（0.86），Sigmoid 核鲁棒性最弱（0.71）；实践中推荐优先尝试 RBF 核，超参数更少且泛化性较强。

PCA 降维实验从两个层次验证了主成分分析的理论。实验 3-1 在 2D 数据上直观展示了降维与重建的过程：保留 1 个主成分时重建误差为 0.3661，恢复点均落在 PC1 直线上；保留 2 个主成分时等价于无损坐标旋转，误差精确为 0，与理论完全吻合。实验 3-2 将 PCA 推广至 590 维的半导体制造数据，结果表明仅需保留 **8** 个主成分（压缩比约 1.4%）即可将相对重建误差控制在 9% 以内，体现了高维工业数据中少数主成分主导绝大部分方差的典型特性。

综合来看，两类算法均高度依赖关键超参数：SVM 需要在核函数类型与 C、d 等参数上仔细调优；PCA 的保留主成分数 N 则需在压缩比与重建精度之间权衡，实践中可参考累积方差解释率或重建误差阈值确定合适的 N。

装 订 线